Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

21/06/2024

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [ ] Zanero [ ] Carminati [ ] Barenghi

Have you done any challenges/homework, even partially? [ ] Yes [ ] No

Do you want to withdraw? [ ] Yes [ ] No

Priority Grading [ ] Yes, Motivation:

[ ] No

Instructions

- **Duration**: 2.5 hours.
- The exam is composed of 19 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**
  - On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
  - You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
  - You are responsible for having all the necessary technological equipment for the correct exam delivery.
  - If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
  - We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
  - Regarding the submission:
    - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
    - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
    - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
    - We will evaluate only the uploaded **readable** documents.
    - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Consider a flight infotainment system that allows passengers to access various entertainment options, such as movies, music, and games, during their flight. Additionally, the system provides internet connectivity via Wi-Fi and allows passengers to register their credit card information for in-flight purchases.

[2 points] 1. Identify and describe two relevant assets at risk in such a scenario, along with their associated threats and threat agents.

Asset: Payment Gateway

- **Threat:** Unauthorized access to credit card information.
- **Threat Agent:** Malicious hackers who exploit vulnerabilities in the system to steal credit card data.

Asset: Quality of Service (QoS) and Reputation

- **Threat:** Denial of Service (DoS) system.
- **Threat Agent:** Disgruntled passengers or individuals seeking to cause disruption.

Additional assets

Asset: Safety ...

Asset: User Internet Traffic ...

[1 point] 2. Suggest, in rough order of importance, the most likely potential threats and threat agents against such infrastructure and their operating companies.

- **1)**
  - **Threat:** Unauthorized access to credit card information. ...
  - **Threat Agent:** Malicious hackers who exploit vulnerabilities in the system to steal credit card data...
- **2)**
  - **Threat:** Denial of Service (DoS) attacks that disrupt the availability of the infotainment system...
  - **Threat Agent:** Disgruntled passengers or individuals seeking to cause disruption...

[1 point] 3. The company proposes an authentication mechanism based on biometrics for accessing the payment system. Is this a good idea? Discuss the advantages and disadvantages of this approach in this specific context.

- **Advantages:**
  - Biometrics, such as fingerprint or facial recognition, can provide a more secure authentication method compared to traditional passwords, as they are difficult to forge or steal.
  - Biometric authentication can be more convenient for users, as they don't need to remember passwords.
- **Disadvantages:**
  - Biometric data is sensitive personal information, and if compromised, it can be difficult to change or revoke.
  - Biometric systems can be expensive to implement and maintain, and may not be accessible to all users, particularly those with disabilities.
  - Biometric systems can be susceptible to spoofing attacks, where an attacker creates a fake biometric to gain unauthorized access.

In the context of a flight infotainment system, the use of biometrics for authentication should be carefully considered, weighing the potential security benefits against the privacy concerns and potential for misuse of biometric data. It's crucial to implement robust security measures to protect biometric data and ensure that the system is resilient to spoofing attacks. Additionally, alternative authentication methods should be provided for users who cannot or do not want to use biometrics.

Exercise 2 - Introduction to Cryptography (6 points)

Consider an instant messaging service, run as a single web-application, where you want to provide message confidentiality and integrity, plus endpoint authentication. Consider the asymmetric encryption scheme in use to be encrypting any 1024 bit string into a 1024 bit ciphertext. All 1024 bit strings are valid plaintexts and ciphertexts. Encryption and decryption keys have the same format.
The users are trusting the messaging service provider not to be impersonating them. The server generates an encryption key pair for each user, at user enrollment time; and sends it to the user, while keeping a public billboard page where the pairs (username, public encryption key).
To initiate a conversation from A to B, user A draws a random bitstring of 1000 bit, appends 24 zero-valued bits to it, applies the decryption algorithm with her own private key, encrypts the result with the B's public key (taken from the billboard) and sends it to B.
Upon receiving the message, B decrypts it with her own private key, and checks that the message is coming from A by encrypting it with A's public key and testing if the last 24 bit are zero-valued. If the check passes, B will use the digest of a secure hash of the retrieved message, without the last 24 bit, as the symmetric key for the session. The symmetric key is then used to encrypt all messages and compute/verify a per-message MAC tag.

[3 points] Analyze the protocol and describe, if existing, a strategy to impersonate A by an adversary E, or argue why the protocol prevents impersonation.

E can impersonate A in the following way:

- Draw a random 1024 bit string, encrypt it with A's public key and test if the last 24 bit are null; if not, try again until they are (8M attempts on average, a couple of minutes on a reasonable computer)
- Take the result of the encryption, remove the trailing 24 bit and compute the digest via the secure hash, obtaining the session key
- Take the drawn random, encrypt it with B's public key and send the result to B

[3 points] Consider the previous protocol, where the asymmetric encryption algorithm is replaced by an asymmetric signature algorithm, having asymmetric signature replace decryption and asymmetric verification replace encryption.. Does the protocol provide impersonation resistance? Is the choice a viable one?

Replacing decryption with signature effectively prevents E from impersonating A, as she will not be able to forge a signature for an arbitrary message she did not see before. Furthermore, the message space is too large (2^1024) to be accidentally having a message signed twice in practice.

The choice is not a viable one however, as having signature and verification replace encryption for in the second use in the protocol would require A to be performing a signature verification action on a message which was never in B's possession, and therefore, cannot check out.

Exercise 3 - Binary Vulnerabilities (6 points)

```
01  #include <stdio.h>
02  #include <string.h>
03  #include <unistd.h>
04  #include <stdlib.h>
05
06  typedef struct{
07      char name[40];
08      char address[60];
09      int age;
10  } person;
11
12  void ascii_art(char *filename){
13      FILE *file;
14      char c;
15
16      file = fopen(filename, "r");
17      if (file){
18          while ((c = getc(file)) != EOF){
19              printf("%c", c);
20          }
21          fclose(file);
22      }
23  }
24
25  void amazing(){
26      person p;
27
28      printf("Enter your name: \n");
29      scanf("%39s", p.name);
30
31      printf("Enter your address: \n");
32      scanf("%59s", p.address);
33
34      if (strlen(p.address) == 59){
35          printf("You entered a very long address: ");
36          printf(p.address);
37          return;
38      }
39
40      printf("Enter your age: \n");
41      scanf("%d", &p.age);
42
43      printf("Hello %s.\n", p.name);
44      printf("You are %d years old.\n", p.age);
45      printf("You live at %s.\n", p.address);
46  }
47
48  int main(){
49      int choice;
50
51      clearenv();
52      ascii_art("banner.txt");
53      printf("Welcome to the most amazing program ever!\n");
54      amazing();
55  }
```

*The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **`cdecl`** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).*

[1 point] 1. Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

Vulnerability	**Line**	**If the vulnerability is present, motivate and modify the line to solve the detected vulnerability**
Buffer Overflow	**No [0.5]**	**Not Present**
Format String	**[0.25] Line 36**	**See Slides [0.25]**

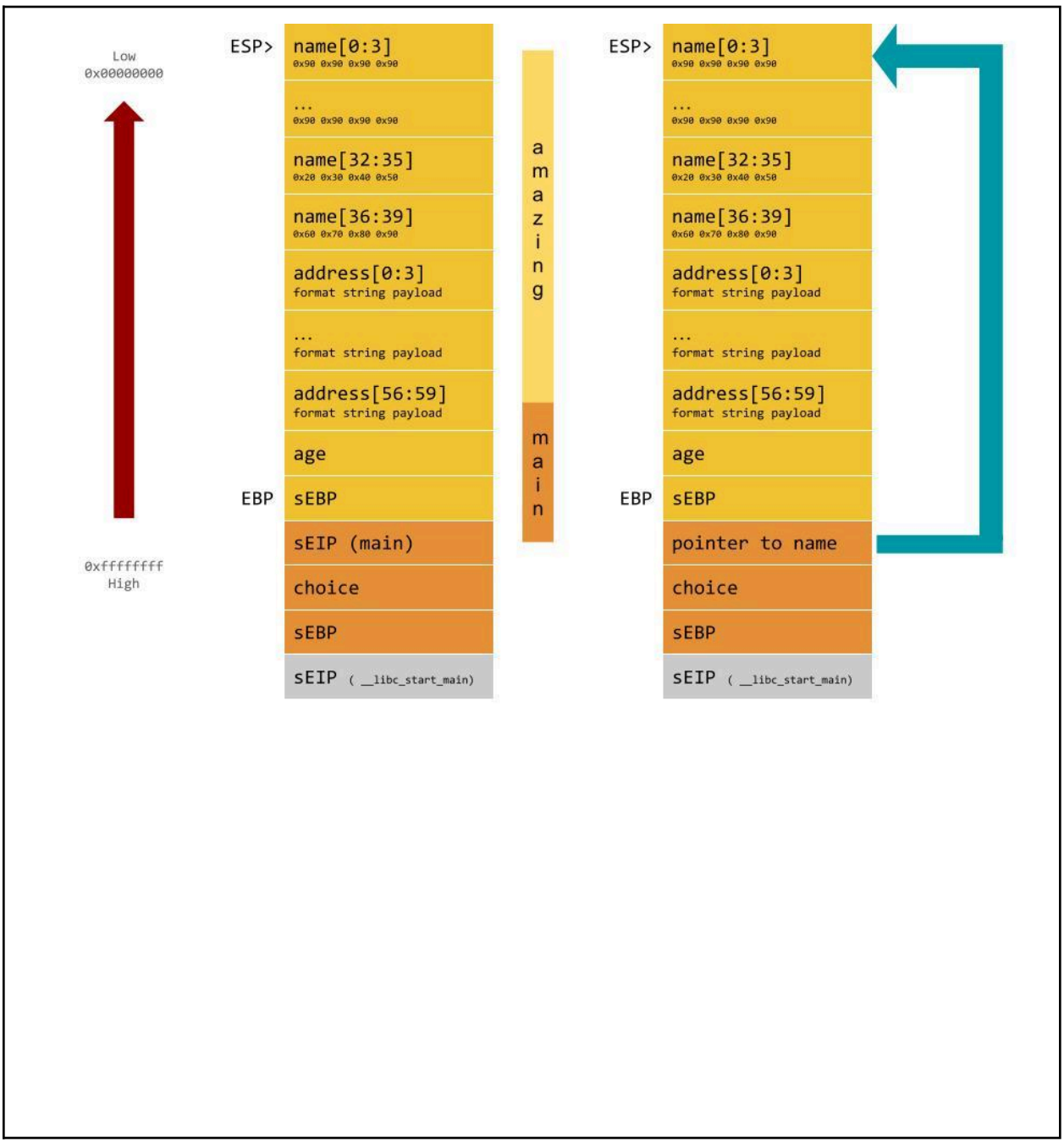
[2 points] 2. Write an exploit that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: `0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90`.

2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

You can put the shellcode inside the variable name or the address. Then, you can exploit the format string vulnerability at line 36 to overwrite the saved EIP with the address of the buffer containing the shellcode. You must insert an address that is 59 bytes long. ASLR and NX must be disabled unless you make further assumptions.

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.



[1 points] 3. Consider ONLY the correctly implemented "Stack Canary" mitigation technique (i.e, randomly generated and non-guessable). Is it effective to prevent your exploit at point 2? Motivate your answer.

The Stack Canary mitigation technique is effective against buffer overflow (BOF) vulnerabilities. However, since we are exploiting a format string vulnerability, this mitigation is not effective, and the exploit at point 2 is still valid.

[2 points] 4. Consider ONLY the correctly implemented "NX" mitigation technique. A friend of yours managed to interact with the program to read the content of the file "./flag" **without executing any shellcode, or recurring to ret2libc**. The interaction was remote, she did not have access to the local machine. If this is possible, please explain **all the steps and assumptions** required for a successful exploitation.

The exploit is possible and you can use the same format string at line 36. Instead of overwriting the return address with the address of the shellcode, you can overwrite it with the address of ascii_art (which is in the .text section). It is important that the argument "./flag" is passed correctly to the function [see slides].

Exercise 4 - Web Vulnerabilities (6 points)

ChatterChaos is a web service for group chatting. Users can view and post messages in channels of personal interest. The relevant code of the application is reported below.

```
00 def blacklist(s):
01     s = s.replace(">", "")
02     s = s.replace("<", "")
03     s = s.replace("[", "")
04     s = s.replace("]", "")
05     return s
06
07 @app.route('/view_channel, methods = ['GET'])
08 def view_channel(request, session):
09     if not is_session_valid(session):
10         return redirect(url_for("/login"))
11     if "channel_id" not in request.keys():
12         return render_template("error.html", error="No page provided!")
13     channel_id = request["channel_id"]
14     if not is_numeric(channel_id):
15         return render_template("home.html", error="Channel id must be numeric!")
16     username = session["username"]
17     view_query = "SELECT username, message, color FROM channel_msgs WHERE channel_id = ?;"
18     messages = db_execute_query_with_statement(view_query, channel_id)
19     return render_template_string(channel_page.html, messages=messages)
20
21 @app.route('/post_message, methods = ['POST'])
22 def post_message(request, session):
23     if not is_session_valid(session):
24         return redirect(url_for("/login"))
25     if "channel_id" not in request.keys():
26         return render_template("home.html", error="No channel specified!")
27     channel_id = request["channel_id"]
28     if not is_numeric(channel_id):
29         return render_template("home.html", error="Channel id must be numeric!")
30     if "message" not in request.keys():
31         return render_template("home.html", error="No message specified!")
32     message = blacklist(request["message"])
33     if "color" not in request.keys():
34         color = "gray"
35     else:
36         color = blacklist(request["color"])
37     username = session["username"]
38     post_query = "INSERT INTO channel_msgs VALUES (?,?,?,?);"
39     result = db_execute_query_with_statement(post_query, channel_id, username,
           message, color)
40     return redirect(url_for("/view_channel"))
…
50 <html>
       …
80     {% for message in messages %} # Iterating over messages
81         <h7>{{ blacklist(message["username"]) }} wrote: </h7> # Displaying username
82         <p style="color:{{ message["color"] }};" onclick="font-weight:bold">
83            {{ message["message"] }}
84         </p>
85     {% endfor %}
       …
100 </html>
```

The relational database schema used by the website is the following:

Table: **users**

username (Primary Key) (String)	hashed_password (String)	email (String)

Table: **channel_msgs**

channel_id (Primary Key) (Integer)	username (String)	message (String)	color

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT … ; INSERT INTO … ;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.
- The username is checked with the function **blacklist** available in the code
- The function **is_numeric** returns true if the string contains only numbers, false otherwise.

More details:

- The function **render_template** generates an HTML page from a .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL of the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided url.
- The function **db_execute_query_with_statement** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, result[4]["username"] is accessing the field `"comment"` of the 5th row.
- The HTML attribute **onclick** specifies the javascript code to be executed when its HTML tag is clicked. Similarly, other attributes, such as **onmouseover**, **onmouseenter**, **onmouseleave**, and **onload**, execute the specified javascript according to their events.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*
SQL Injection (SQL-I)	*No*	*Nothing to do*
cross-site scripting (XSS) **Specify if reflected, stored, DOM...**	*Lines 81, 82, and 83. The user can perform a stored XSS. The most realistic input is the color since it doesn't need any angular bracket*	*Whitelisting + Escaping (see slides)*
Cross-site request forgery (CSRF)	*Yes, the function post_message performs a state-changing action, and there's no CSRF token*	*Use CSRF tokens (see slides)*

[1.5 points] 2. After visiting a streaming website, you notice that somebody posted several messages on many channels with your username. These messages advertise a cryptocurrency service that you don't even know. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The exploit is a CSRF attack, and the visited page contains a form that is auto-submitted as soon as the page is loaded by the browser.
The form is the following:

```
<form method="POST" action="/post_message">
  <input type="hidden" name="channel_id" value="100">
  <input type="hidden" name="message" value="You want some easy money! Visit
      easycrypto.com!">
  <input type="submit" onload=click()>
</form>
```

[1.5 points] 3. The channel dedicated to animal discussion has been hacked. Every time you visit the page of the channel, a popup appears saying, *"Who's the king of the jungle!!!"*. Explain all the steps you would take to reproduce the exploit. The final code does not need to be syntactically correct, but it has to point where is the vulnerability and the idea of the exploit.

The vulnerability is an XSS in the color field:

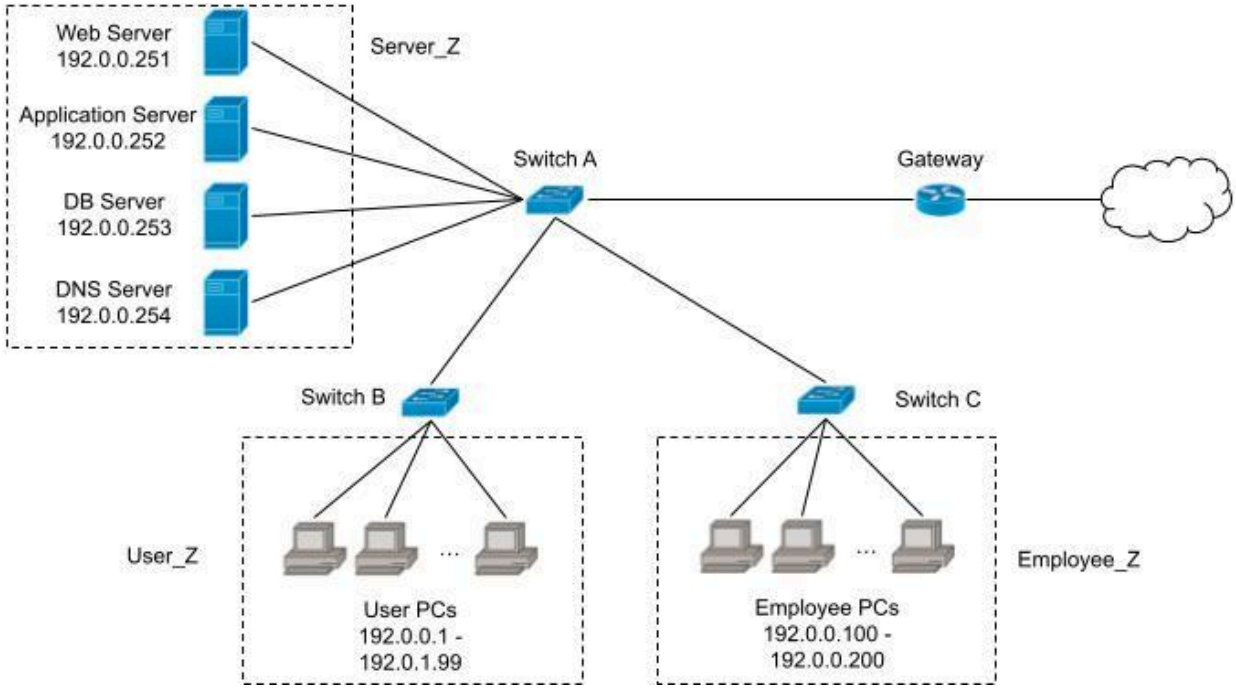
```
Color = black" onload="alert('Who's the king of the jungle!!!')
```

Exercise 5 - Network Security (6 points)

Eat&Play is a new gaming bar that wants to offer a high-quality gaming experience to its customers at a reasonable price. They allow users to book one or multiple one-hour timeslots in advance and assign each new user a unique username and password to access their PCs. When users log in, they have a wide variety of games they can play, and they can also log in to their Steam account to perform microtransactions inside the games.
Eat&Play has its own internal infrastructure, which is composed of computers that can be rented by the customers, computers for the employees, and a few servers:

- A web server, that hosts the booking website
- An application server, that hosts the business logic of the service
- A DB server, which is used to manage bookings and users
- A DNS server.

The network layout is schematized in the following picture.



One day, a user complained that their credit card information had been stolen after one of their gaming sessions, and someone made expensive purchases with them. Obviously, the system administrator has to investigate this accident: while going through the logs of switch A, they find a suspicious sequence of packets.

Timestamp	Src IP:port	Dst IP:port	Description
+00,000s	*192.0.0.2:1234*	*DNS_IP:53*	*Recursive DNS query for "www.buyvbucks.com"*
+00,001s	*DNS_IP:2345*	*Auth_DNS_IP:53*	*Recursive DNS query for "www.buyvbucks.com"*
+00,002s	*5.61.75.45:5432*	*DNS_IP:1024*	*DNS response: "www.buyvbucks.com" is 192.0.0.6*

+00,003s	*5.61.75.45:5432*	*DNS_IP:1025*	*DNS response: "www.buyvbucks.com" is 192.0.0.6*
...	...	...	...
+00,103s	*5.61.75.45:5432*	*DNS_IP:2345*	*DNS response: "www.buyvbucks.com" is 192.0.0.6*
+00,201s	*DNS_IP:16342*	*192.0.0.2:1234*	*DNS response: "www.buyvbucks.com" is 192.0.0.6*
+00,312s	*192.0.0.2:62345*	*192.0.0.6:443*	*HTTPS request to "www.buyvbucks.com"*
+10,010s	*Auth_DNS_IP:6548*	*DNS_IP:2345*	*DNS response: "www.buyvbucks.com" is 84.65.45.154*

[1 point] 1. Can you identify what is going on? State the name of the attack and how it works.

The attack is DNS cache poisoning.
[see slides for explanation]

[1 point] 2. Who is the attacker? Can their IP be spoofed? Who is the victim?

The attacker is a customer.
Their IP is spoofed in the requests, but they must provide at least one "true" IP address to which the traffic will be redirected.
*Therefore, in this situation, the IP is [**192.0.0.6**].*

[3 points] 3. To prevent this accident from happening again, the owner of *Eat&Play* hired you as a security consultant to modify the network layout by inserting a firewall. You have to choose where to insert it: you can remove one of the switches and replace it with a firewall. Where would you place the firewall?

Switch A

Write the firewall rules you deem necessary, making them consistent with your choice and assuming you are working with a stateful packet filter.

Firewall	Src IP	SRC Port	Direction	Dst IP	Dst PORT	Policy	Description
FW1	*all*	*any*	*all*	*all*	*all*	*Deny*	*Default Deny*

FW1	*all*	*any*	*Internet -> Server_Z*	*WS_IP*	*80/443*	*Allow*	*Allow connection to webserver*
FW1	*User_IP*	*any*	*User_Z -> Server_Z*	*DNS_IP*	*53*	*Allow*	*Allow DNS requests*
FW1	*Employee_IP*	*any*	*Employee_Z -> Server Z*	*DNS_IP*	*53*	*Allow*	*Allow DNS requests*
FW1	*Employee_IP*	*any*	*Employee_Z-> Server_Z*	*WS_IP*	*80/443*	*Allow*	*Allow admin access to WS*
FW1	*User_IP*	*any*	*User_Z -> Internet*	*any*	*80/443*	*Allow*	*Enable internet connection for users*
FW1	*Employee_IP*	*any*	*Employee_Z -> Internet*	*any*	*80/443*	*Allow*	*Enable internet connection for employees*
FW1	*Employee_IP*	*any*	*Employee_Z -> Server_Z*	*any*	*22*	*Allow*	*Enable SSH connection to servers for employees*

[1 point] 4. Is there another way the attacker could obtain the same result? If yes, how? If not, why?

Yes, the attacker could poison the DNS cache of the victim PC by answering the first DNS request (the one coming from the victim) directly instead of poisoning the cache of the DNS server.

Exercise 6 - Malware (6 points)

The information system of Penacony, the Planet of Festivities, has been infected with malware, potentially compromising the success of the Charmony Festival, the greatest festival that will soon take place in Penacony. Fortunately, it seems that the malware has not yet harmed the system. Sunday, the head of the Oak Family, one of the most influential families in Penacony, was able to get a copy of the malware, and is asking you to analyze it.

Here is some information to analyze the malware. Suppose the malware has been written to work on Linux systems.

- **void *signal(int sig, void * fun)** associates the function *fun* with the signal number *sig*. When the OS passes the signal *sig* to the program, the function *fun* will be executed.
- **int raise(int sig)** makes the OS send the signal sig.
- SIGTRAP is a signal used by debuggers to set and handle breakpoints.
- By default, debuggers don't pass signals to the program, but they intercept signals before they reach the program. You can set debuggers to pass signals to the program, but then your debugger won't be able to work anymore, e.g., they won't intercept SIGTRAPs.
- The Charmony Festival starts on the 22/06/2024.

```
00      void bad_stuff() {
01              // Here the program does something bad
02              puts("Encrypting filesystem...");
03      }

04      int main() {
05              // Register the signal handler SIGTRAP -> bad_stuff
06              signal(SIGTRAP, bad_stuff);

07              // Unix timestamp of 22/06/2024
08              time_t t1 = 1719007200;
09              time_t t2 = t1 - time(NULL);

10              sleep(t2);

11              // Raise the SIGTRAP signal
12              raise(SIGTRAP);

13              puts("Done!");
14      }
```

[3 points] 1. Which stealth techniques does this malware employ? Please explain how **they** are implemented in this particular case. Please note: giving vague answers and naming wrong techniques will make you lose points.

Time bomb until charmony festival (see the sleep at line 10) and anti-debugging technique. While running outside a debugger, raise(SIGTRAP) provokes the execution of bad_stuff. While running inside a debugger, the execution is stopped at raise(SIGTRAP) and bad_stuff will never be called.

[2 points] 2. What type of analysis would you apply to the analyzed malware? Please provide a detailed motivation of your answer and specify all the assumptions you have made.

Considering the time bomb and the anti debugging, you have to apply static analysis.

While analyzing the traffic, Sunday found another variant of the malware, and asked you to analyze it.

```
void bad_stuff() {
	// Here the program does something bad
	puts("Encrypting filesystem...");
}

// Function which should initialize
// the malware, as in the previous sample
void init_program() {
}

int main() {
	void * init_program_base = (void *) init_program;
	for (int i = 0; i < 0x100; i++) {
	*((char *) init_program_base + i) ^= 0x43;
	}
	init_program();
}
```

3) [1 points] Compared to the original version, what new stealth technique does this malware implement? Which analysis technique would you use? Why? Please specify all the assumptions you have made. Please note: giving vague answers and naming wrong techniques will make you lose points.

Polymorphism. The malware decrypts the init_program function and then executes it. You have two choices:

- Statically decrypt the init_program function, and then apply static analysis to the malware
- Use dynamic analysis to decrypt init_program, dump the decrypted binary, and then use static analysis as for the point 1.